

Efficient Memory Management

CS1040 Program Construction

Eng Prof Chandana Gamage

BSc Eng Hons, MEng, PhD, CEng

Department of Computer Science & Engineering
University of Moratuwa

February 3, 2026

Memory Management

Memory management is a critical aspect of program execution that is controlled by the operating system and the programming language used to code the program.

- ▶ Program languages with their libraries and run-time execution environment provides mechanisms for allocating and deallocating memory during program execution.
- ▶ Key activities of memory management are:
 - ▶ Memory allocation
 - ▶ Memory deallocation
 - ▶ Garbage collection
 - ▶ Use of pointers for memory access
 - ▶ Ensuring memory safety

Memory Allocation Techniques

Memory is requested by a program from the OS when it needs to store data, which can be allocated statically (at compile time) or dynamically (at runtime).

	Explicit Allocation	Implicit Allocation
Explicit Deallocation	C language, malloc(), free()	
Implicit Deallocation	Java language, new operator, gc thread	Python language, gc thread gc thread

- ▶ Memory, when no longer required by the program, needs to be deallocated and returned to the OS.
- ▶ Non-deallocated memory results in memory leaks.
- ▶ Memory leaks slow down program execution and ultimately lead to program crashes.

Garbage Collection

Programming languages, such as Java, Python, Go, and JavaScript have automated memory management through a process called garbage collection.

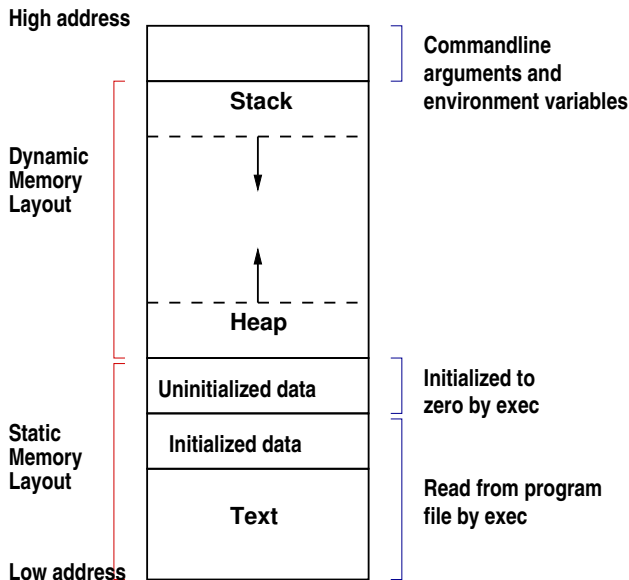
- ▶ Garbage collectors automate the process of memory management in programming languages by automatically detecting and freeing memory that is no longer in use.
- ▶ Eliminates error-prone and time-consuming work where programmers need to manually allocate and deallocate memory.
- ▶ Garbage collectors trace through a program's data structures to find all the objects that are still reachable from active variables or from the stack.
- ▶ Any memory that is not reachable, or that is only reachable through other unreachable memory, is considered garbage.

Garbage Collection Techniques

Garbage collection algorithm typically uses a combination of reference counting and tracing to determine which memory is still in use.

- ▶ **Reference counting** keeps track of the number of references to each object.
 - ▶ An object is marked as garbage when the reference count decrements to zero.
- ▶ **Tracing** use a set of *memory roots* (which is typically the table of active variables and the stack) and recursively follow all the pointers to other objects in memory.
 - ▶ An object is marked as garbage when it is not reached during the tracing process.

Memory Layout for C programs



Memory Allocation in C 1/3

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc , char *argv[]) {
    int i = 1; // allocated from initialized data segment
    int j; // allocated from uninitialized data segment
    int *ptr; // allocated from heap segment
    ptr = malloc(sizeof(int)); // allocate memory
    printf("%i %i %i\n",i,j,*ptr);
    i = 2;
    *ptr = 3;
    printf("%i %i %i\n",i,j,*ptr);
    j = 4;
    free(ptr); // deallocate memory
    printf("%i %i %i\n",i,j,*ptr);
}
```

Memory Allocation in C 2/3

```
void func2(int *ptr1) { // pointer pass-by-value
    int *ptr2;
    printf("%i %i\n",*ptr1,*ptr2);
    ptr1 = malloc(sizeof(int)); // allocate memory
    ptr2 = malloc(sizeof(int)); // allocate memory
    printf("%i %i\n",*ptr1,*ptr2);
}
```

```
void test2() {
    int *ptr1; // allocated from heap segment
    printf("%i\n",*ptr1);
    func2(ptr1);
    printf("%i\n",*ptr1);
    free(ptr1); // deallocate memory
    printf("%i\n",*ptr1);
}
```


Memory Allocation in C 3/3

```
void func3(int **ptr1) { // pointer pass-by-reference
    int *ptr2;
    printf("%i %i\n",**ptr1,*ptr2);
    (*ptr1) = malloc(sizeof(int)); // allocate memory
    ptr2 = malloc(sizeof(int)); // allocate memory
    printf("%i %i\n",**ptr1,*ptr2);
}
```

```
void test3() {
    int *ptr1; // allocated from heap segment
    printf("%i\n",*ptr1);
    func3(&ptr1);
    printf("%i\n",*ptr1);
    free(ptr1); // deallocate memory
    printf("%i\n",*ptr1);
}
```

Memory Allocation in Java

```
class MemoryTest {  
    private int i;  
  
    public MemoryTest(int i) {  
        this.i = i;  
    }  
  
    public void print() {  
        System.out.println(i);  
    }  
  
    public static void main(String [] args){  
        MemoryTest mt;  
        mt = new MemoryTest(1040);  
        mt.print();  
    }  
}
```

Efficient Use of Memory in Java

- ▶ To achieve memory efficiency:
 1. Minimize the **amount of memory** used by a program and the computational cost of allocation
 2. Minimize the **amount of work** the garbage collector has to do and reduce the cost of deallocation
- ▶ **Caching** is the best technique to reduce computational cost of allocation/garbage collection
- ▶ Caching increase the memory footprint of the program
- ▶ Software engineers have to make a trade-off between memory footprint size and allocation/garbage collection cost

The new Operator in Java

Use the new operator to create an **object** from a **class**

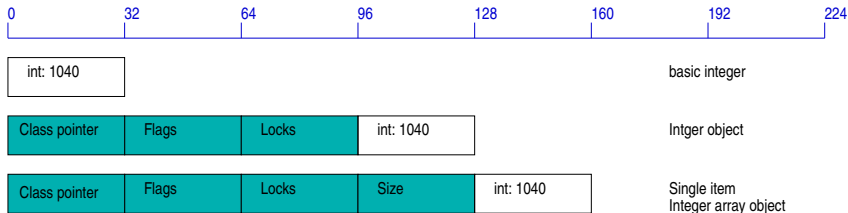
- ▶ Class is a *program template* giving details of **state** and **methods**
 - ▶ State is represented by a set of variables with access usually restricted to the object itself (**private**)
 - ▶ Methods are a set of functions with some accessible only by the object (**private** helper methods) and others accessible by other objects (**public** API calls)

```
public static void main(String[] args) {  
    int i = 1040; // a basic integer type variable  
    Integer intObj = new Integer(1040); // an Integer object  
}
```

- ▶ **Quiz:** On a 32-bit system, how much memory is required for `i` and `intObj`?

Internal Structure of Java Objects 1/2

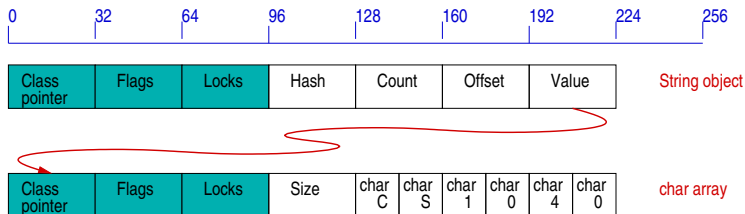
- ▶ Typical object metadata
 - ▶ Class pointer - pointer to class information, for example, Integer or String
 - ▶ Flags - object shape, hash code, etc
 - ▶ Lock - lock variable or a pointer to a *monitor* object used to control concurrent access
 - ▶ Size - the length of the array for array-type objects
- ▶ `int` variable to Integer object size ratio is **1:4**



Internal Structure of Java Objects 2/2

- ▶ The java.lang.String object **encapsulates** a character array
- ▶ A **fixed memory overhead** of 352 bits is added to any stored string
- ▶ Short string objects give a very high overhead ratio

```
public static void main(String[] args) {  
    // a java.lang.String object  
    String strObj = new String("CS1040");  
}
```



Garbage Collector

- ▶ Most objects created by an application are **temporary**
 - ▶ Used to store data for calculations, visualization/presentation, storage/transmission, etc
 - ▶ Number of objects required could be determined statically (at compile-time) or dynamically (at run-time)
- ▶ Temporary objects need memory to be allocated for their creation
 - ▶ Allocated memory must be reclaimed when no longer required
 - ▶ Done when the *thread of execution* moves out of the **scope** of a variable representing an object
- ▶ Memory management carried out by the Garbage Collector (*gc*) process
 - ▶ *gc* executes as a separate thread
 - ▶ *gc* is an overhead on the application performance
 - ▶ Overhead on execution time due to *gc* is typically up to 10%

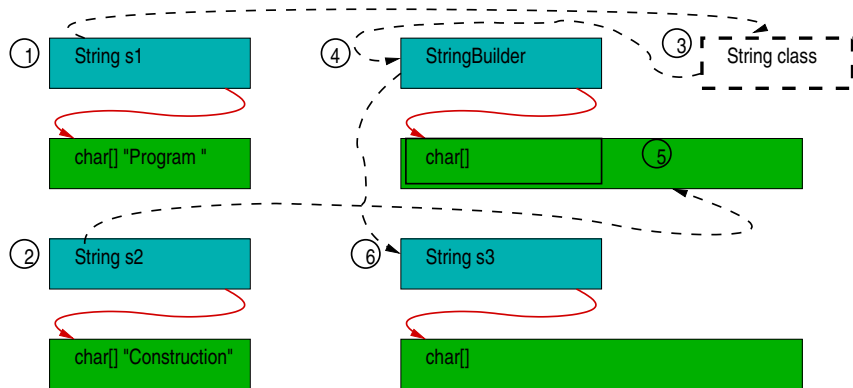
String Concatenation and Temporary Objects 1/3

```
public static void main(String[] args) {  
    // explicit string object creation  
    String s1 = new String("Program ");  
    // implicit string object creation  
    String s2 = "Construction";  
    // string concatenation using + operator  
    String s3 = s1 + s2;  
}
```

- Concatenation of string values in the two string objects using the `+` operator and returning a string object equates to:

```
String s3 = new  
    StringBuilder(String.valueOf(s1)).append(s2).toString()
```


String Concatenation and Temporary Objects 2/3



String Concatenation and Temporary Objects 3/3

1. String s1 is created at compile-time
2. String s2 is created at compile-time
3. In class String, the **method**
`static String valueOf(Object obj)`
returns the string representation of the Object argument
4. In class StringBuilder, the **constructor**
`StringBuilder(String str)`
constructs a string builder initialized to the contents of the specified string
5. In class StringBuilder, the **method**
`StringBuilder append(String str)`
appends the specified string to this character sequence
6. In class StringBuilder, the **method**
`String toString()`
returns a string representing the data in this sequence

String Interning and String Literals

- ▶ String *interning* is a method of storing only one copy of each unique String value, which must be immutable
 - ▶ This is an optimization technique done by the Java compiler
 - ▶ Interned strings are stored in a cache-like data collection by the JVM
 - ▶ String *literals* such as `s1` and `s2` are automatically interned by the compiler
- ▶ Prevents memory allocation for Strings that are already in the String intern cache by providing references
- ▶ Concatenation of final String literals (when using `+` operator) is done at compile-time
- ▶ **Quiz:** How does the JVM *efficiently* determine if a literal string is already stored in the cache?

Summary

- ▶ An object is much larger than the item(s) of data stored in it
- ▶ An object encapsulates a data item allowing Java run-time to provide
 - ▶ Memory management
 - ▶ Data synchronization
- ▶ It also allows (debugging) tools to analyze and visualize application
- ▶ A 64-bit system would have a greater negative effect on memory usage
- ▶ **Scope** of variables referencing objects must be carefully handled to ensure performance efficiency of *gc* process
- ▶ Memory allocation and deallocation can be reduced using caching with only referencing required
- ▶ Overall memory requirement can be reduced with caching by sharing rather than copying